Augury: A Polyglot Pipeline for Testing Whether Social Momentum Leads Prediction Markets

Design, Numerical Guarantees, and Cross-Language Verification

Matthew Park

July 28, 2026

Abstract

Prediction markets aggregate beliefs into a price. Social media aggregates the same beliefs into text. If the text moves first, a liquid market is leaving information on the table; if it does not, the market is efficient with respect to that signal. Augury is a five-language pipeline built to distinguish those two cases without accidentally manufacturing the first. It streams posts from X, filters near-duplicates with streaming MinHash and LSH, extracts *target-conditioned stance* via natural-language inference rather than sentiment, aggregates it into a time-decayed signal $S(t)$, calibrates a Hanson LMSR market maker against real order-book depth, and evaluates lead–lag structure under stationarity testing and false-discovery-rate correction. This paper documents the design and, more importantly, the verification: the shared mathematics is implemented three times and pinned to 53 golden vectors at 10^{-9}, of which two implementations are currently executed; the numerically fragile paths are analysed rather than asserted. That analysis produces three corrections to the system's own documented claims — the float64 underflow threshold for the decay kernel is $\lambda\Delta t > 745$, not the 233 claimed; the shipped LSH band configuration retrieves only 20.4% of true duplicates at its own declared threshold; and the per-step liquidity recalibration described in the simulation docstring is provably a no-op as implemented. No lead–lag result is reported, because none is yet earned: the ingester has not run long enough. The contribution is the instrument and its calibration, not a finding.

1 Introduction

The hypothesis under test is that public social sentiment carries information a liquid prediction market has not already priced. Efficient markets should mostly falsify it. That asymmetry shapes every design decision below: a pipeline built to find a relationship will find one, because there are enough degrees of freedom between raw text and a p-value to reach almost any conclusion. The harder engineering problem is building a pipeline that can cleanly report *no relationship*, and that is what this system is for.

Concretely, the failure modes that produce spurious lead–lag findings are well known and individually unremarkable:

- joining a stale price to fresh sentiment across a data outage;
- running a Granger test on two integrated series;
- testing forty markets at $p < 0.05$ and reporting the two that fired;
- scoring a signal in $[-1, 1]$ against an outcome in $\{0, 1\}$ with a squared-error rule;

Table 1: Services, responsibilities, and verification status. Line counts cover all git-tracked files of each type, test suites included.

Service	Language	LOC	Responsibility	Status
`augury-signal`	Python 3.12	7,545	NLP, $S(t)$, orchestration	runs, 139 tests
`augury-ingest`	Rust 1.97	2,469	Streaming ingest, filter	not compiled
`augury-engine`	C++20	1,470	LMSR, walk-forward	not compiled
`augury-api`	Java 25	1,222	REST, WebSocket, ledger	9 tests (JDK 21)
`augury-analytics`	R 4.6	981	Granger, Brier, reports	runs, 55 tests
Schema / migrations	SQL, JSON	1,200	Shared contracts	migrations applied

- letting a spam ring of fifty accounts count as fifty independent opinions;
- choosing a market-maker liquidity parameter until the simulated path looks convincing.

Each of these is guarded against in code rather than left to the analyst's discipline. Section 7 covers the first four, Section 5 the fifth, and Section 6 the sixth.

Contributions.

1. **A polyglot architecture with explicit contracts** (Section 2): five services in Rust, Python, C++, Java, and R, joined by a shared TimescaleDB schema, JSON Schema payload contracts, and a canonical market identifier, with Redis as a convenience rather than a dependency.
2. **A numerically defensible decay kernel** (Section 3), evaluated in log space with the peak factored out, together with a precise account of where float64 actually fails — which is not where the code comments claim.
3. **Depth-calibrated LMSR liquidity** (Section 6): the market-maker parameter b is derived in closed form from the observed bid–ask width and resting depth, and the derivation exposes a 723× sensitivity to which side of a real, lopsided book is used.
4. **An LSH banding analysis** (Section 5) showing the shipped configuration is mistuned relative to its own similarity threshold, with the asymmetric cost argument that fixes it.
5. **Cross-language verification** (Section 8): 53 golden vectors generated from the Python reference and asserted at 10^{-9} by independent implementations in R and C++, with the vectors chosen to be hostile rather than representative.
6. **An honest verification ledger** (Sections 10 and 12), separating what has been executed from what has only been written.

2 Architecture

Five services divide the work along the axis of what each language is actually good at (Table 1). Ingestion is I/O-bound and must not lose data, so it is async Rust. Modelling is library-bound, so it is Python. The simulation core runs the LMSR step function tens of thousands of times per backtest sweep, so it is C++. Serving is a concurrency problem, so it is reactive Java. Econometrics belongs where the econometrics libraries are, so it is R.

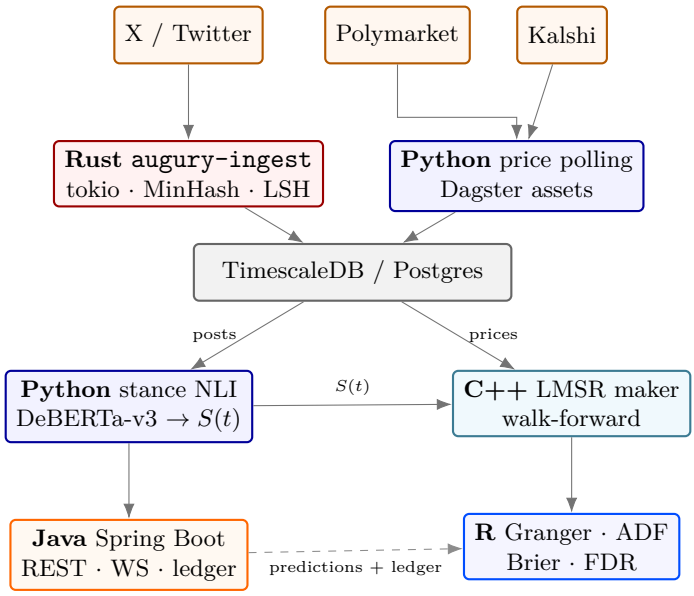

Figure 1: Data flow. Sources at the top, the durable record in the middle, modelling and simulation below it, serving and econometrics at the bottom. Every arrow crosses either the database or a schema-validated Redis channel; no service holds a reference to another.

2.1 Cross-service contracts

Services never call each other directly. Two boundaries carry everything:

TimescaleDB is the durable record. A market is keyed everywhere as `<venue>:<ticker>` (for example `kalshi:KXFEDDECISION-26SEP-C25`), which is what keeps cross-venue joins unambiguous — Kalshi tickers and Polymarket condition ids share no namespace. Hypertable partitioning forces a schema property worth noting: any unique constraint must include the partitioning column, which is why `posts` is keyed on $(\texttt{post\_id}, \texttt{market\_id}, \texttt{created\_at})$ rather than `post_id` alone. That is not merely a concession to Timescale; one post can legitimately be evidence about two markets, and the compound key says so.

Redis pub/sub carries live updates on `augury.signal.*`, `augury.price.*`, and `augury.sim.*`. Payloads are validated against JSON Schema *at the publisher*. This is the only place the contract can be enforced: the C++ and Java consumers are separately compiled and cannot be type-checked against the Python publisher, so a schema check at the boundary is the sole mechanism preventing silent drift. Redis is deliberately not a dependency — an outage degrades the live stream while ingestion, scoring, and the REST API keep working off the database.

3 The Social Signal $S(t)$

Raw sentiment is the wrong quantity for a prediction market. What matters is a post's *stance* toward a specific claim, $\text{stance}_i \in [-1, 1]$. The aggregate signal at time t is a weighted average under exponential decay:

$$S(t) \;=\; \frac{\sum_{i=1}^{N(t)} w_i \,\text{stance}_i \, e^{-\lambda(t-t_i)}}{\sum_{i=1}^{N(t)} w_i \, e^{-\lambda(t-t_i)}}, \qquad w_i = \ln(1+f_i)\,(1+e_i), \qquad \lambda = \frac{\ln 2}{t_{1/2}}, \tag{1}$$

Table 2: Decay kernel magnitude by post age and half-life. Underflow to exactly zero begins at $\lambda\Delta t > 745.1$.

Half-life	Post age	$\lambda\Delta t$	$e^{-\lambda\Delta t}$
6 h	7 d	19.41	3.73×10^{-9}
6 h	30 d	83.18	7.52×10^{-37}
30 min	7 d	232.90	7.14×10^{-102}
30 min	22.4 d	745.27	**0** (underflow)
30 min	30 d	998.13	**0** (underflow)

with f_i followers and e_i engagements on post i. The logarithm on followers is the usual reach compression; the linear factor on engagements credits amplification.

3.1 Log-space evaluation, and where float64 actually fails

Both sums in Eq. 1 are evaluated in log space with the maximum term factored out. Writing $\ell_i = \ln w_i - \lambda(t - t_i)$ and $m = \max_i \ell_i$,

$$S(t) \;=\; \frac{\sum_i \text{stance}_i \, e^{\ell_i - m}}{\sum_i e^{\ell_i - m}}, \tag{2}$$

so the dominant term is $e^0 = 1$ and the common factor e^m — which may be entirely unrepresentable — cancels before it is ever formed.

The decay module's docstring motivates this by claiming that at a 30-minute half-life a week-old post carries e^{-233}, "which underflows to exactly zero in float64." **That is not correct**, and the correction is worth stating because it changes where the guard actually binds. IEEE-754 binary64 carries subnormals down to $\approx 4.94 \times 10^{-324}$, so exp returns zero only for arguments below $\ln(4.94 \times 10^{-324}) \approx -745.1$. Concretely, $e^{-233} = 7.14 \times 10^{-102}$, which is an entirely ordinary normalised double. Table 2 gives the real boundary.

The guard is still necessary, and the shipped test data proves it. The golden vector named `extreme_age` evaluates a 30-day-old post at a 30-minute half-life, giving $\lambda\Delta t = 998$. Its recorded expectations are $s_t = 0.5$ — the stance of the only post present, which Eq. 2 recovers correctly — alongside `weight_sum` $= 0.0$ *exactly*. That second field is the underflow itself, pinned as a fact in the test data: the absolute weight has vanished while the ratio remains well defined. A naive implementation would compute 0/0 here. What the code does is right; only the stated threshold was off, by a factor of three in the conservative direction, and a reader calibrating a shorter half-life from the documented figure would have over-estimated the danger zone.

3.2 Two degenerate cases handled rather than papered over

A zero-follower account gets $\ln(1) = 0$ and therefore contributes *nothing*, no matter how much engagement the post drew. That is what Eq. 1 says, so it is what the implementation does — but it means a window containing only zero-follower posts has no defined signal. In that case, and when the window is empty, `compute_signal` returns `None` rather than 0.0.

The distinction is load-bearing. Zero is a real, neutral signal value. Returning it for "nobody has said anything" would tell the downstream LMSR that the crowd is balanced when in fact the crowd is silent, and the market maker would then trade on a claim nobody made. Every consumer is required to propagate the absence rather than substitute a default.

Separately, posts timestamped after the evaluation instant are excluded unconditionally. This is the single cheapest place to introduce lookahead into a backtest, and it is precisely the bug that makes a lead–lag result look spectacular and be worthless.

3.3 Adaptive half-lives

A fixed six-hour half-life means that during an FOMC statement or a debate, posts from before the news still dominate the average. The effective half-life is therefore shortened in proportion to how unusual the current window is, under two independent triggers, whichever is stronger:

$$t_{1/2}^{\text{eff}} = \max\left(t_{\min},\ \frac{t_{1/2}}{\max\left(1,\ \frac{n_{\text{recent}}}{\kappa\, r_{\text{base}}},\ \frac{|\Delta p|}{\theta}\right)}\right), \tag{3}$$

with κ the volume-surge multiple (default 3), θ the price-move threshold (default 0.05), and $t_{\min}$ a configurable floor (default 30 minutes). Below both triggers the factor is exactly 1, so a quiet market keeps its baseline half-life bit-for-bit.

Two details matter. The baseline rate r_{base} is the *median* hourly post count over a trailing 24 hours, not the mean: one viral hour would otherwise raise the baseline enough to mask the next surge, which is exactly when the shortened half-life is wanted. And that median is floored at one post per hour, because most tracked markets are quiet most of the time and a median of zero would make the surge ratio undefined — silently disabling the trigger in the case where it is most informative, a quiet market that suddenly receives thirty posts.

Because the half-life varies, it is recorded per signal point in the `signals` table rather than assumed constant. A series whose decay parameter moved is not comparable to one whose did not, and the column is what makes that recoverable after the fact.

4 Target-Conditioned Stance

Standard sentiment analysis fails on prediction-market phrasing in a specific and systematic way. "Candidate X drops out" is negative *text sentiment* and strongly *bullish stance* toward "Will Candidate Y win?". A lexicon scores the words and gets the sign backwards.

Augury encodes each post and the market's target claim as a natural-language inference sentence pair — premise = post, hypothesis = claim — and derives

$$\text{stance} = P(\text{entail}) - P(\text{contradict}), \qquad \text{confidence} = 1 - P(\text{neutral}). \tag{4}$$

This mapping has a property worth stating: a post the model finds neutral receives a stance near zero *and* a low confidence, so it neither pushes the signal nor claims to be informative. The two quantities degrade together, which is what one wants from an uncertainty estimate.

Consequently every tracked market carries a declarative `target` in `config/markets.yaml` — a claim, never a question, because entailment against an interrogative is ill-defined and the outputs drift toward neutral. The market titled "Will the Federal Reserve cut rates by 25bps at their September 2026 meeting?" carries the target "The Federal Reserve will cut interest rates by 25 basis points at its September 2026 meeting."

Three implementation choices are worth recording. First, NLI label names vary across checkpoints (`ENTAILMENT`, `entailment`, `LABEL_0`), so the model's own `id2label` map is inspected by regular expression rather than assumed, and a checkpoint whose entailment and contradiction indices cannot be identified raises instead of guessing. Second, weight loading is lazy, so constructing

the model is free and the several hundred megabytes are pulled only when something is actually scored — which keeps the test suite and the `markets` subcommand from downloading a model they will never use. Third, the checkpoint default points at an NLI-tuned model, because plain `deberta-v3-base` is a pretrained encoder with a *randomly initialised* classification head: it has no concept of entailment, and using it directly would produce confident noise rather than an error.

A VADER lexicon model is retained as the ensemble's second opinion. It cannot, by construction, condition on the target, so it is not a competitor — it is a disagreement detector. The per-post quantity $|\text{stance}_{\text{NLI}} - \text{stance}_{\text{VADER}}|$ is stored as a data-quality flag, on the reasoning that a post the target-conditioned model reads as bullish and the lexicon reads as bearish is either the exact case the NLI framing exists to handle, or a post ambiguous enough to distrust.

5 Ingestion and Near-Duplicate Filtering

A spam ring posting the same claim from fifty accounts with rotated links and minor edits moves $S(t)$ as much as fifty genuine opinions. Exact hashing catches none of it.

5.1 MinHash and LSH

For a random permutation of the shingle universe, the probability that two sets share a minimum element is exactly their Jaccard similarity; averaging that indicator over H independent permutations gives an unbiased estimate with standard error $\approx 1/\sqrt{H}$. The implementation uses word trigrams over normalised text, $H = 128$ permutations drawn from the universal family $h_i(x) = (a_i x + b_i) \bmod p$ with $p = 2^{61} - 1$, and a deterministic splitmix64 seed — deterministic because signatures are persisted to `BIGINT[]` columns and compared against posts hashed by a later process run on a different machine.

Normalisation strips URLs, mentions, cashtags, and hashtags. This is the correct direction for an adversarial filter: a spam ring rotates its links and handles while reusing the body, so *keeping* those tokens would make otherwise identical posts look distinct.

Algorithm 1 has two properties chosen deliberately. Duplicates are pushed onto the window along with originals, so a ring posting twenty variants matches against the whole cluster rather than only its first member. And the window is bounded — an unbounded index grows without limit over a long ingest run, and comparing against month-old posts is not useful anyway, since the same phrasing recurring next week is a separate event rather than a duplicate. The bound is what makes this a *streaming* filter.

5.2 The banding is mistuned

Splitting an H-permutation signature into b bands of r rows makes two documents with Jaccard similarity s collide in at least one band with probability

$$P_{\text{compare}}(s) \;=\; 1 - (1 - s^r)^b. \tag{5}$$

The shipped configuration is $H = 128$, $b = 8$, $r = 16$, with a duplicate threshold of $\tau = 0.80$. A source comment states that this "crosses 50% around Jaccard 0.80." It does not. Solving Eq. 5 numerically, the 50% point for $(b, r) = (8, 16)$ is $s = 0.856$, and at the declared threshold $\tau = 0.80$ the probability that a genuine duplicate pair is even *compared* is 20.4% (Table 3, Figure 2). Roughly four in five true duplicates at threshold are never examined.

Algorithm 1 Streaming near-duplicate check against a bounded window

Require: post (id, text), window W (capacity C), threshold τ
Ensure: verdict, and the signature to persist

$\sigma \leftarrow \textsc{MinHash}(\text{text})$
if $\sigma = \perp$ **then** ▷ no shingles: URLs and handles only
  return $(\textsc{Unique}, \perp)$ ▷ cannot be a duplicate of anything
end if
$B \leftarrow \textsc{Buckets}(\sigma)$ ▷ one LSH key per band
$best \leftarrow \perp$
for $s \in W$ **do**
  if $B \cap s.\text{buckets} = \emptyset$ **then continue** ▷ LSH pre-filter
  end if
  $j \leftarrow \textsc{Similarity}(\sigma, s.\sigma)$ ▷ exact signature agreement
  if $j \geq \tau$ **and** $(best = \perp$ **or** $j > best.j)$ **then**
    $best \leftarrow (s, j)$
  end if
end for
if $|W| = C$ **then**
  evict oldest from W
end if
push (id, σ, B) onto W ▷ duplicates are remembered too
return $(best = \perp \ ? \ \textsc{Unique} : \textsc{Duplicate}(best), \sigma)$

Table 3: Probability that a pair with Jaccard similarity s is retrieved for comparison. The shipped configuration's steep region sits to the *right* of the $\tau = 0.80$ decision threshold, which is the wrong side.

s	0.60	0.70	0.75	**0.80**	0.85	0.90	0.95
shipped (b=8, r=16)	0.002	0.026	0.077	**0.204**	0.461	0.806	0.990
proposed (b=16, r=8)	0.237	0.613	0.815	**0.947**	0.994	1.000	1.000

The fix follows from an asymmetry in the costs. LSH here is purely a *pre-filter*: every retrieved candidate is then subjected to an exact signature comparison against τ, so a false positive from banding costs one 128-element integer comparison and nothing else, while a false negative is a duplicate that silently enters $S(t)$ as an independent opinion. The banding should therefore be tuned so its steep region sits to the *left* of the decision threshold. Repartitioning the same 128 permutations as $b = 16$ bands of $r = 8$ rows moves the 50% point to $s = 0.674$ and lifts retrieval at $\tau = 0.80$ to 94.7%, at the price of examining 23.7% of pairs at $s = 0.60$ — all of which the exact check then rejects. No additional storage or hashing is required; it is a repartitioning of an existing signature.

5.3 Rejects are stored, not dropped

Rejected posts are written to the database with their verdict (`duplicate`, `bot`, `low_quality`) and a human-readable reason, alongside their MinHash signature. Without the rejects there is no way to measure how often the filter is wrong, and a filter nobody can audit is a filter nobody should trust. The signal query reads through a partial index (`WHERE filter_verdict = 'accepted'`), so

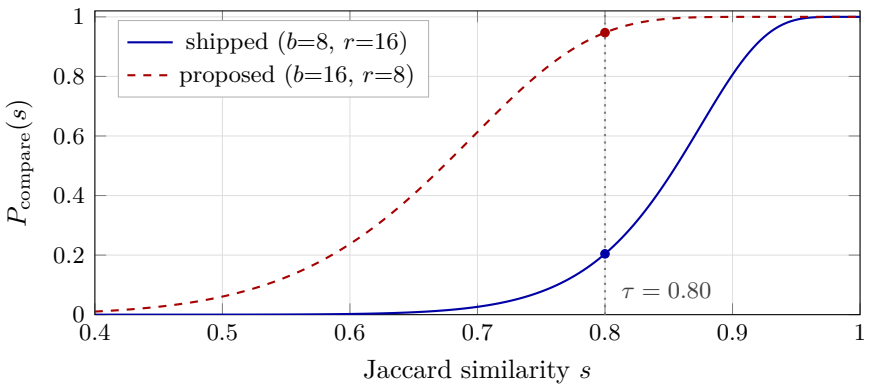

Figure 2: LSH retrieval probability. The decision threshold is $\tau = 0.80$; the shipped banding retrieves only 20.4% of true duplicates there.

retaining the rejects costs storage but not query time.

The heuristic thresholds are deliberately loose — account age under 14 days *and* fewer than 50 followers, text under 12 characters, symbol ratio above 0.6, a small promotional-marker list — on the reasoning that wrongly dropping a real opinion silently biases the signal, whereas letting a little junk through adds noise that the follower weighting already discounts. A young account with a real audience is a person, not a ring, and passes.

5.4 The read budget

X charges roughly $0.005 per post read, so the budget is the only thing between a polling loop and a real bill. Two properties are non-negotiable. It must survive restarts, because an in-process counter resets on every crash and a crash loop then becomes unbounded spending; the counter therefore lives in a `read_budget` table keyed by (UTC day, service). And it must be reserved *before* the request goes out, because a request that times out after the server has already metered it would otherwise slip through unbilled.

The reservation is a single statement so that two concurrent workers cannot both observe headroom and both spend it:

```
INSERT INTO read_budget (day, service, reads_used)
SELECT CURRENT_DATE, $1, $2 WHERE $2 <= $3
ON CONFLICT (day, service) DO UPDATE
    SET reads_used = read_budget.reads_used + EXCLUDED.reads_used
    WHERE read_budget.reads_used + EXCLUDED.reads_used <= $3
RETURNING reads_used
```

Both guards are required, and the reason is a defect this codebase shipped with before review. On the first request of a UTC day there is no conflicting row, so `ON CONFLICT` never fires — and the natural `INSERT ... VALUES` form would happily insert a single reservation larger than the entire daily budget. The `WHERE` clause on the `SELECT` closes that hole; the `ON CONFLICT` guard covers every subsequent request. Unused reservations are refunded after the response arrives, without which the effective ceiling would collapse from the number of *reads* to the number of *requests*.

Finally, anything derived from replayed fixtures carries `source='fixture'` into the database, so a number computed from replayed data can never later be mistaken for a real measurement.

Table 4: Depth calibration on a live Kalshi book: bid 0.03×93, ask $0.04 \times 67{,}296$. The logit width is 0.2980. The choice of side changes b by a factor of 723.

Side used	Depth (shares)	b	Worst-case subsidy $b \ln 2$
bid only	93.0	312	\$216
mean	33,694.6	113,052	\$78,362
ask only	67,296.1	225,792	\$156,507

6 The LMSR Synthetic Market

Hanson's Logarithmic Market Scoring Rule defines a cost function and the price vector implied by its gradient:

$$C(\mathbf{q}) = b \ln\Big(\sum_{j=1}^{K} e^{q_j/b}\Big), \qquad p_k(\mathbf{q}) = \frac{e^{q_k/b}}{\sum_j e^{q_j/b}}, \qquad \text{cost}(\mathbf{q} \to \mathbf{q} + d\mathbf{q}) = C(\mathbf{q} + d\mathbf{q}) - C(\mathbf{q}). \quad (6)$$

For a binary market the structure collapses to a sigmoid, $p_{\text{yes}} = \sigma\big((q_{\text{yes}} - q_{\text{no}})/b\big)$, hence

$$\text{logit}(p_{\text{yes}}) = \frac{q_{\text{yes}} - q_{\text{no}}}{b}, \quad (7)$$

so buying n YES shares moves $\text{logit}(p)$ by exactly n/b. That linearity is what makes b interpretable as liquidity and is the basis of everything below.

6.1 Calibrating b from observed depth

The liquidity parameter controls both the market maker's worst-case subsidy $b \ln K$ and how far a single trade moves the price. Chosen arbitrarily, the synthetic market either barely reacts to the signal or whipsaws on every post, and tuning it until the output looks convincing is exactly the degree of freedom that makes a simulation unfalsifiable. It is therefore derived.

In a real book, consuming the size resting between the bid and the ask walks the price across the quoted range. By Eq. 7, moving the synthetic price the same distance costs $b\,(\text{logit}(a) - \text{logit}(d))$ shares for bid d and ask a. Setting the two equal:

$$b = \frac{\text{depth}}{\text{logit}(a) - \text{logit}(d)}. \quad (8)$$

Recalibrating b between trades deliberately holds the current price fixed: by Eq. 7, preserving p under a change of b means rescaling the share imbalance by the same ratio. This is a modelling commitment, not an implementation convenience — a liquidity update is not new information about the outcome, and letting it move the price would inject signal that nobody traded on.

6.2 Real books are severely lopsided

Equation 8 leaves one choice open: which side's depth. On a symmetric book it does not matter. Real prediction-market books are not symmetric. Table 4 applies Eq. 8 to a live Kalshi snapshot of `KXFEDDECISION-26SEP-C25` taken during the run in Section 10.

A 723× spread in the single parameter that governs how responsive the simulated market is cannot be resolved by picking a default and moving on. The default here averages both sides,

which is symmetric but not obviously correct; the honest reading of Table 4 is that on a deeply out-of-the-money market the ask side is thick because nobody wants the position at all, and the bid side is thin for the same reason, so neither number is measuring "liquidity" in the sense the model assumes. This is recorded as an open question rather than a solved one, and `side` is a first-class parameter in all three implementations so that a sensitivity sweep is a configuration change rather than a rewrite.

6.3 Numerical stability, stated precisely

Every exponential in all three implementations goes through a max-subtracted log-sum-exp. The source comments justify this by asserting that "q_j/b reaches several hundred and `exp` overflows to infinity on entirely ordinary inputs." The precise statement is that binary64 overflows for arguments above $\ln(1.798 \times 10^{308}) = 709.78$. The golden vector chosen to exercise this path uses $q/b = 500$, which does *not* overflow ($e^{500} = 1.40 \times 10^{217}$). The guard nevertheless binds in reachable configurations: b is clamped to a floor of 1.0, at which a position of 710 shares is sufficient to overflow, and the K-outcome price vector computes $e^{q_j/b} / \sum_j e^{q_j/b}$, which becomes $\infty/\infty = \text{NaN}$ rather than a merely inaccurate number. Log-sum-exp is the right implementation; the comment overstates how easily the failure is reached, and a reader who trusted it would draw the wrong conclusion about which test inputs are stressful.

6.4 Walk-forward backtesting

A single train/test split answers one question about one arbitrary cut point. The C++ engine rolls the split instead: fit on window N, evaluate on window $N + 1$, advance by the test size so that test windows never overlap. Overlapping windows would reuse observations across evaluations and make the results look more consistent than they are.

Within each window, b is calibrated from the *training* window's book only — using the test window's depth would leak information the simulation could not have had — and the responsiveness parameter is grid-searched on the training window. Two reported quantities are structured to make a negative result reachable:

- Every window's MSE is reported against a **random-walk baseline** that simply predicts the last price observed before the test window began. When fewer than half the windows beat it, the summary prints, in so many words, that the signal adds nothing over the market's own last print and that this is a result rather than a bug.
- The **coefficient of variation of** b across windows is reported as a first-class output, with a warning above 50%. A b that swings by an order of magnitude between adjacent windows means the depth calibration is tracking noise, and every MSE figure computed on top of it is irreproducible. Given Table 4, this is not a hypothetical concern.

The engine is deliberately file-in/file-out over JSON with no database or Redis client. The numerical core is the part worth writing in C++; keeping the I/O boundary at JSON means the binary is trivially testable and has no transport dependencies to break. A separate benchmark target measures per-operation cost, kept out of `ctest` because a benchmark has no pass/fail condition and wiring one into CI makes the suite fail whenever a build agent is busy.

7 Econometrics

The question is whether $S(t)$ carries information about where $P(t)$ goes next, beyond what P's own history already implies. The vector autoregression is

$$P_t = \alpha_0 + \sum_{j=1}^{p} \alpha_j P_{t-j} + \sum_{j=1}^{p} \beta_j S_{t-j} + \epsilon_t, \qquad H_0 : \beta_1 = \cdots = \beta_p = 0. \tag{9}$$

Four guards stand between a naive version of this test and a defensible answer, and all four are enforced in code.

Logit transform. $P_t \in [0, 1]$ and $S_t \in [-1, 1]$ are both bounded. A market at 0.03 cannot fall much further, so its innovations are asymmetric and its level is near-certainly non-stationary in the sense the test assumes. Both series are mapped through $\tilde{x} = \ln(x/(1-x))$ before fitting, with S first mapped to $(S+1)/2$ so it can share the transform. Lag order p is chosen by AIC or BIC, capped at $\min(12, \lfloor (n-2)/4 \rfloor)$ so the VAR is never fit on fewer effective observations than it has coefficients.

Stationarity is tested, not assumed. An Augmented Dickey–Fuller result travels with every reported test, for both series. Granger causality between two integrated series manufactures impressive p-values out of pure spurious regression. The ADF null is a unit root, so a *small* p-value means stationary — the opposite of the intuition most readers carry into it, which is why the R implementation returns the result as a named list rather than a bare number.

Alignment is a fixed convention, not a parameter. Lead–lag results are extremely sensitive to how the two series are joined. Post time is `created_at`, price time is bar close, both are resampled onto the same regular grid by last-value-in-bar, and a value is carried across *at most one* empty bar. A loose `merge_asof` tolerance — as the Phase 1 prototype used, at two hours — will pair a stale price with fresh sentiment and produce a relationship that is an artifact of the join.

This convention is also where the deepest defect in the codebase was found. The R implementation originally bucketed timestamps with `cut()`, which returns a *factor* whose levels are wall-clock strings carrying no timezone; `as.POSIXct()` then reinterpreted each label in the session's local zone and silently shifted every timestamp by the UTC offset, so the signal/price join matched almost nothing. The covering test asserted only an upper bound on row count, so it passed at zero rows. The fix is integer arithmetic on epoch seconds, which is exact and zone-independent.

Multiple comparisons. Running this test once per market across dozens of markets at $p < 0.05$ produces roughly one false "sentiment leads price" finding per twenty by chance. A Benjamini–Hochberg FDR correction is applied across the whole batch, and the `significant` field is defined off the adjusted p-value *only* — it is `NA` until the correction runs, and an uncorrected result reports as not significant, because in isolation it is not yet an answer. The serving layer enforces the same discipline: the `/granger` endpoint returns only the newest `run_id`, since mixing runs means mixing p-values corrected against different batches.

The reverse direction (`price_to_signal`) is a first-class option throughout, because a signal that merely *follows* price is the null result this project should be able to report cleanly.

7.1 Forecast calibration

When a market resolves at time T to outcome $O \in \{0, 1\}$, accuracy is scored by the Brier score $\mathrm{BS} = \frac{1}{N}\sum_t (\hat{p}_t - O)^2$.

Note the $\hat{p}_t$, not $S(t)$. The signal ranges over $[-1, 1]$ and is not a probability; scored directly, a maximally bearish signal $S = -1$ against an outcome of 0 — a *correct* call — would contribute $(-1 - 0)^2 = 1$, the worst possible value. The signal is mapped into probability space first, and which mapping was used is stored alongside every score, because scores computed under different mappings are not comparable quantities. Two mappings exist: **affine**, $\hat{p} = (S + 1)/2$, assumption-free and always available; and **Platt**, $\hat{p} = \sigma(a + bS)$, fit by Newton–Raphson on the two-parameter logistic over resolved markets.

The Platt fit targets *smoothed* labels following Platt's original formulation, $y^+ = (N^+ + 1)/(N^+ + 2)$ and $y^- = 1/(N^- + 2)$. This is not cosmetic. Augury will have a handful of resolved markets long before it has many, and small samples are frequently linearly separable — every market the signal called bullish happened to resolve YES. Against raw 0/1 labels the maximum-likelihood solution for separable data is unbounded: Newton drives b toward infinity, log loss toward zero, and the resulting "calibrator" reports probabilities of exactly 1.0. A model whose entire purpose is tempering overconfidence would instead be manufacturing it. The reported loss is computed against the true 0/1 outcomes rather than the smoothed targets, since smoothing is a fitting device and quoting the smoothed loss would flatter the model. The fit refuses to run at all when every resolved market in the batch had the same outcome, on the grounds that such a batch says nothing about where the decision boundary sits.

Reported alone, the Brier score still answers the wrong question: a market that traded at 0.02 for a month and resolved NO yields a superb score for any forecast that also said "no". The reported quantity is therefore the Brier Skill Score against the market's own price at the same timestamps:

$$\mathrm{BSS} = 1 - \frac{\mathrm{BS}_{\mathrm{signal}}}{\mathrm{BS}_{\mathrm{market}}}. \tag{10}$$

$\mathrm{BSS} > 0$ means the X-derived signal is better calibrated than the market price itself — the actual claim this project exists to support or refute. $\mathrm{BSS} \leq 0$ is an equally useful result: it says the market is already efficient with respect to this signal. Both branches are implemented, named, and printed in plain language.

8 Cross-Language Verification

The same mathematics is implemented three times: in Python (the reference), in C++ (the simulation engine), and in R (the analytics module). Independent implementations of a formula agree only by luck unless something forces the issue.

The forcing function is `schemas/testdata/golden_vectors.json`, generated from the Python reference by `python -m augury_signal.golden` at a fixed epoch so the file is byte-identical between runs. It holds 53 cases (Table 5), emitted at full float64 repr precision; the consuming suites compare at 10^{-9}, loose enough to absorb differences in summation order and tight enough that a genuine formula divergence cannot hide.

Python and R currently agree to 10^{-9} across the whole file. The C++ suite asserts the same vectors and will close the loop once the service compiles (Section 12). Regenerating after an

Table 5: Golden-vector inventory. Each case fixes inputs and the expected output of the Python reference.

Group	Covers	Cases	Notable case
`signal`	$S(t)$ aggregation	7	`extreme_age`: weight underflows to 0
`lmsr.cost`	$C(\mathbf{q})$, price vector	6	$q/b = 500$; $K = 4$ and $K = 3$
`lmsr.trade`	$C(\mathbf{q} + d\mathbf{q}) - C(\mathbf{q})$	3	buys and sells
`lmsr.binary_price`	sigmoid identity	3	$q_{\text{yes}} = -900$
`lmsr.shares_to_move`	$b(\text{logit}\, p' - \text{logit}\, p)$	3	$0.97 \to 0.03$
`lmsr.b_calibration`	Eq. 8	5	the real Kalshi book; a crossed book
`adaptive_decay`	effective half-life	5	floored at $t_{\min}$
`calibration`	affine, logit, sigmoid	21	logit(0.001): boundary clamp
Total		**53**	

intentional change to the reference mathematics is expected to break the other suites, and that breakage is the mechanism working rather than a nuisance to be routed around.

Two design details make this gate load-bearing rather than decorative. The vectors are chosen to be hostile rather than representative — a 30-day-old post at a 30-minute half-life whose weight underflows to zero, a crossed book that must fall back rather than raise, $q/b = 500$ in the cost function, and logit inputs of 0.001 and 0.999 that exercise the boundary clamp instead of diverging. And the C++ suite locates the file by walking up from the build directory, so the gate cannot be accidentally skipped by a build layout change; a missing file fails the suite rather than silently passing zero assertions.

9 Serving Layer

The Java service is a reactive Spring Boot gateway over the same schema, plus the paper-trading ledger. Three decisions are worth recording.

Blocking work never touches the event loop. Every JDBC call is wrapped so that it runs on the bounded-elastic scheduler rather than the reactor thread:

```
Mono.fromCallable(work).subscribeOn(Schedulers.boundedElastic())
```

Blocking a Netty worker thread stalls every other request that thread is multiplexing; with a small worker pool that is a service-wide outage rather than one slow response.

SQL NULL is not zero. `ResultSet.getDouble` returns 0.0 for SQL `NULL`, and for a price that is a meaningful and wrong value — 0.0 means "certain NO", not "unknown". Every optional numeric goes through a `wasNull()`-checked accessor. The same principle governs the valuation path: a position with no available mark returns a null price and an explicitly incomplete valuation rather than an unrealised P&L of zero, which would read as "flat".

Optional filters need explicit casts. PostgreSQL infers a parameter's type from how it is used, and a parameter appearing *only* inside `? IS NULL` gives it nothing to infer from; the server rejects the statement with "could not determine data type of parameter". Every optional filter is written `CAST(? AS text) IS NULL OR col = ?`. This was a shipped defect: the unit tests mock

Table 6: Vertical slice, `kalshi:KXFEDDECISION-26SEP-C25`, 28 July 2026. Prices and order book are live Kalshi data; posts are fixture replay.

Stage	Result
Price history	278 hourly bars fetched
Order book	bid 0.03×93, ask $0.04 \times 67{,}296$, spread 0.0100
Calibrated b	113,052 (mean side; see Table 4)
Posts	12, fixture replay, `source='fixture'`
Stance model	`vader-3.3.2` (lexicon baseline)
$S(t)$ series	49 hourly points; latest $S = -0.559$, $\hat{p} = 0.221$
Effective half-life	6.00 h (no surge trigger fired)
LMSR path	$0.147 \to 0.220$ over 49 steps
Real market mid	0.035 (gap +0.185)
Strongest $\lvert r \rvert$	+0.658 at lag −3 h — *price leads*
Granger $S \to P$	$F = 1.003$, $p = 0.463$, lag 7 (AIC), $n = 39$
ADF p-values	signal 0.144 (unit root not rejected), price 0.025
Calibration	market unresolved; Brier/BSS deferred

the repository, so the SQL was never executed against a server, and the failure appears only at runtime.

The paper-trading ledger holds no venue credentials of any kind, which is an architectural boundary rather than an unfinished feature. Its sizing rule is worth stating because it encodes the project's thesis directly: the target position is proportional to the signal's *edge over the market* $\hat{p} - p_{\text{market}}$, not to the signal level. A signal saying "80% likely" is worth nothing when the market already says 80%; it is informative only when it disagrees. Sizing on the level alone would pile into consensus positions and mistake agreement for insight.

10 A Worked End-to-End Run

The `slice` subcommand executes the entire chain in memory — poll, ingest, score, aggregate, simulate, analyse — with no database and no billable API calls. Table 6 reports a live run against `kalshi:KXFEDDECISION-26SEP-C25` on 28 July 2026.

What this run demonstrates and what it does not are different things, and the tool says so itself: the report terminates with an explicit notice that posts were replayed and the numbers are not evidence about the real relationship between X and this market.

Read as an instrument check, several things are working correctly. The cross-correlation peak sits at lag −3 h with the interpretation label *price leads* — the direction that falsifies the project's hypothesis, reported as prominently as the direction that would confirm it. The Granger test does not reject at any conventional level, and the report immediately annotates that this is an uncorrected p-value which requires Benjamini–Hochberg adjustment across every market before it means anything. And the ADF test fails to reject a unit root in the signal series at $p = 0.144$ — which is not evidence that the series *is* integrated, but is exactly the condition under which the F statistic printed two lines above it should not be trusted. The guard fired and said so rather than being silently satisfied.

Two things visible here are genuinely problematic and worth naming. The simulated LMSR price ends at 0.220 against a real market mid of 0.035, a gap of 18.5 probability points. That gap is not evidence of predictive power; it is the affine mapping $\hat{p} = (S + 1)/2$ doing exactly what Section 7 predicts it will do on an out-of-the-money market. A moderately bearish lexicon signal

Table 7: Defects found by review, and why the test suite missed each.

Location	Defect	Why tests missed it
R `align_on_grid`	`cut()` dropped the timezone, shifting every timestamp by the local UTC offset; the join matched almost nothing.	The covering test asserted only an upper bound on row count, so it passed at zero rows.
Java `AuguryRepository`	Optional filters used a bare `? IS NULL`, which PostgreSQL rejects with "could not determine data type of parameter".	Unit tests mock the repository, so the SQL was never sent to a server.
Rust `ReadBudget::reserve`	The `ON CONFLICT` ceiling guard does not fire on the first insert of a UTC day, so one request larger than the whole daily budget was allowed through.	The path requires a live database and an empty budget table; integration tests skip without `DATABASE_URL`.
C++ engine	Five missing standard headers (`<cstdint>`, `<limits>`, `<string>`, `<utility>`, `<stdexcept>`).	Invisible without a compiler; the service has never been built (Section 12).

of $S = -0.559$ maps to $\hat{p} = 0.221$, roughly six times the market's own price, because the affine map's implicit claim that $S = 0$ means 50/50 is a strong and here plainly false assumption. This is precisely the case Platt calibration exists to correct, and it cannot be fit until markets resolve. The second is that this run used the VADER lexicon rather than the NLI model, so the stance scores are sentiment, not stance, and the sign is untrustworthy on exactly the phrasing Section 4 describes.

11 Defects Found by Review

A full correctness review was run over the codebase. Table 7 records what it found. Each entry is a class of bug the existing tests could not have caught, which is the property that makes them worth recording rather than silently fixing.

11.1 A fifth defect, recorded as fixed but still present

The review recorded a fifth item — that Python's `simulate_lmsr` "recalibrated b to the same constant every step, making per-step liquidity tracking a no-op, and left a dead local behind" — and marked it resolved. It is not resolved, and the distinction matters because the function's own docstring makes a claim the implementation does not support.

The docstring states that "b is re-calibrated at each step from the most recent book snapshot, so the synthetic market's sensitivity tracks the real market's liquidity instead of being a constant chosen to make the output look good." In the implementation, b is computed once from a single book snapshot before the loop begins, and the in-loop call is `market.recalibrate(b)` with that same unchanged value.

That call is a no-op, and the proof takes two steps. By the invariance property of Section 6, recalibration rescales the share imbalance by $b_{\text{new}}/b_{\text{old}}$ to hold $\text{logit}(p) = (q_{\text{yes}} - q_{\text{no}})/b$ fixed; with b unchanged that ratio is exactly 1. The method then canonicalises the state by assigning the whole imbalance to the YES leg, which is *not* in general an identity — but along this code path it is, because every mutation of the market goes through `buy_yes`, which touches only q_{yes}, so q_{no} is identically zero from construction onward. Both steps are therefore identities, and the call

Table 8: Test inventory. Uncompiled code is unverified code, and is labelled as such.

Suite	Framework	Count	Status
`augury-signal`	pytest	139 tests	**pass** (1.3 s, ruff clean)
`augury-analytics`	testthat	55 expectations	**pass**
`augury-api`	JUnit	9 tests	passed under JDK 21; POM now targets 25
`augury-ingest`	cargo	42 tests	**never compiled**
`augury-engine`	Catch2	26 cases	**never compiled**

cannot change any field of the market state. The dead local likewise survives: it is now explicitly discarded with `del ordered_prices` at the end of the function, which silences the linter without removing the dead computation.

There is a deeper reason the docstring cannot be satisfied as written. The price polling step appends exactly *one* order-book snapshot per cycle — history comes from candles, which carry bid and ask but no depth — so there is no per-step book history to calibrate against even in principle. The correct version of this behaviour already exists in the C++ backtester, which calibrates b per walk-forward window from that window's own training-period books and holds it fixed through the test period. Bringing the Python path in line means either persisting a depth series or restating the docstring to describe what the data supports. Both are legitimate; silently disagreeing with the docstring is not, which is why it is reported here rather than quietly patched.

12 Verification Status and Limitations

Table 8 is the honest accounting. Three of five services have been executed on the development machine; two have not, because the toolchain to do so is unavailable there — Rust builds are blocked by Windows Smart App Control, which blocks the unsigned `build.rs` executables Cargo compiles and runs, and no C++ compiler is installed. Both blockers are environmental and documented with the commands to clear them, but neither has been cleared, and uncompiled code is unverified code. The 42 Rust and 26 C++ test cases in Table 8 are written, not passing.

Beyond that:

- **No lead–lag result exists.** A Granger test on a handful of hourly bars proves nothing. The ingester must run for days to weeks per market before any finding is worth reporting, and the Platt calibration cannot be fit until several markets have actually resolved with both outcomes represented.
- **The stance path needs a real checkpoint in production.** The default is NLI-tuned, which makes zero-shot stance detection work without a labelled corpus, but a fine-tune on prediction-market stance data would very likely beat it — and the run in Section 10 used the lexicon baseline, not the NLI model.
- **Test coverage is uneven.** The pure-mathematical core is thoroughly covered and cross-validated; the database layer, Redis bus, and pipeline orchestration are not, the Java tests cover ledger arithmetic rather than the controllers, and there is no cross-service integration suite. Two of the four defects in Table 7 live in exactly those gaps.
- **Liquidity controls are specified but not wired in.** Depth and spread are collected and stored, and `granger_results.liquidity_control` records whether a run used them — it is

currently always false. Without them, "sentiment leads price" can be an artifact of when the book happened to be thin.

- **The LSH banding fix in Section 5 is analysed, not applied.** Changing it invalidates stored bucket keys, so it requires a migration rather than a constant edit.

13 Conclusion

Augury is an instrument, and this paper is its calibration report. The mathematics it implements — a decay-weighted stance aggregate, a depth-calibrated LMSR market maker, a guarded Granger test, and a market-referenced Brier skill score — is not individually novel. What the system contributes is the discipline around it: three independent implementations pinned to a shared set of hostile golden vectors; a set of statistical guards that are enforced in code rather than in the analyst's memory; and an evaluation path on which the answer "the market is already efficient with respect to this signal" is as easy to reach and as clearly printed as its opposite.

The analysis in this paper produced three corrections to the system's own claims. The float64 underflow threshold for the decay kernel is $\lambda\Delta t > 745$, not 233, so the documented danger zone was off by a factor of three in the conservative direction. The shipped LSH band configuration achieves 20.4% retrieval at its own declared duplicate threshold, which the asymmetric cost structure of a pre-filter says should be closer to 95%; the fix is a repartitioning of an existing signature, not new machinery. And the per-step liquidity recalibration described in the simulation docstring is a provable no-op as implemented, for the structural reason that the pipeline stores only one book snapshot per cycle. None of the three was found by the test suite, and none could have been: two are incorrect *claims* about correct code, which no assertion is written to catch, and the third is a function that satisfies every test while contradicting its own docstring. Two surfaced from working the arithmetic out on paper and checking it against the source; the third from reading the implementation against the behaviour it advertises. That is an argument for writing the paper.

The generalisable point is the one the null result makes available. On a question where the prior should be that the effect does not exist, an apparatus that cannot report its absence is not measuring anything. The guards documented here — FDR correction before significance is defined, a random-walk baseline on every backtest window, a market-referenced skill score, and a b-stability statistic that invalidates its own MSE figures when it drifts — exist so that when the ingester has run long enough to produce an answer, the answer will be worth the wait, in whichever direction it falls.

References

[1] Hanson, R. (2003). Combinatorial information market design. *Information Systems Frontiers*, 5(1), 107–119.

[2] Hanson, R. (2007). Logarithmic market scoring rules for modular combinatorial information aggregation. *Journal of Prediction Markets*, 1(1), 3–15.

[3] Granger, C. W. J. (1969). Investigating causal relations by econometric models and cross-spectral methods. *Econometrica*, 37(3), 424–438.

[4] Benjamini, Y., & Hochberg, Y. (1995). Controlling the false discovery rate: a practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society, Series B*, 57(1), 289–300.

[5] Brier, G. W. (1950). Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1), 1–3.

[6] Platt, J. (1999). Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In *Advances in Large Margin Classifiers*, MIT Press, 61–74.

[7] Broder, A. Z. (1997). On the resemblance and containment of documents. In *Proceedings of the Compression and Complexity of Sequences*, 21–29.

[8] Leskovec, J., Rajaraman, A., & Ullman, J. D. (2020). *Mining of Massive Datasets*, 3rd ed., Ch. 3: Finding similar items. Cambridge University Press.

[9] He, P., Gao, J., & Chen, W. (2023). DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing. *ICLR 2023*.

[10] Yin, W., Hay, J., & Roth, D. (2019). Benchmarking zero-shot text classification: datasets, evaluation and entailment approach. *EMNLP-IJCNLP 2019*, 3914–3923.

[11] Said, S. E., & Dickey, D. A. (1984). Testing for unit roots in autoregressive-moving average models of unknown order. *Biometrika*, 71(3), 599–607.